IntelXtract

Tactical OSINT reconnaissance and reporting platform

AI-Powered OSINT Automation & Intelligence Platform

Local-first collection · Entity correlation · Deterministic risk scoring · Professional reporting

Domain · IP · URL · Email · Username · File — 19 collectors, one platform

ENGINEERING PROJECT REPORT

Architecture, capabilities, quality evidence, and release engineering

Document ID	**IX-ENG-REP-1.0**
Product version	**1.0.0 (tag v1.0.0)**
Document version	**1.0**
Date	**14 October 2026**
Status	**Approved for release**
Classification	**Internal / Public Distribution**
Primary language	**Python 3.13+ (validated on 3.13 / 3.14)**
License	**GPL-3.0-or-later**

This report documents a production software system: architecture, implementation, verification results, and distribution mechanics.

Document Control

Field	Value
Document title	IntelXtract — Engineering Project Report
Document ID / version	IX-ENG-REP-1.0 / 1.0
Product version	1.0.0 (release tag v1.0.0)
Date of issue	14 October 2026
Status	Approved for release
Classification	Internal / Public Distribution
Scope	Full product: architecture, implementation, verification, and distribution
Source of record	Repository source tree, automated test suite outputs, and project documentation
Change control	Superseded only by a versioned re-issue tied to a product release tag
Intended audience	Engineering leadership, security reviewers, technical stakeholders, and contributors

Table 1. Document control summary.

This report is the authoritative engineering record for the IntelXtract 1.0.0 release. It describes what the product does and why it exists, how it is architected and implemented, how it was verified, and how it is packaged and distributed. It is organised so that a reader can stop after Section 1 for the executive picture, read Sections 2–9 for the product and its design, and consult Sections 10–16 for verification evidence, operations, security, and release engineering. Quantitative claims are traceable to the repository.

Reading path	Sections	Purpose
Executive	1	Essential facts, figures, and outcome for decision-makers
Product & design	2–9	Problem, objectives, architecture, engine, modules, analysis, persistence, reporting
Interfaces & operations	10–12	CLI, GUI, assistant, scheduling, cases, plugins
Assurance	13–15	Security & privacy, quality assurance, performance
Delivery	16–19	Packaging, CI/CD, methodology, positioning, roadmap, conclusion
Reference	Appendices	Dependencies, configuration, catalogs, and reproduction commands

Table 2. Suggested reading paths by audience.

Table of Contents

Document Control 2

1 Executive Summary 7

1.1 Delivery summary . . . 7

1.2 Outcome and assessment . . . 7

2 Introduction and Background 9

2.1 What open-source intelligence is . . . 9

2.2 The OSINT landscape and its friction . . . 9

2.3 Motivation . . . 9

3 Problem Statement and Objectives 10

3.1 Problem statement . . . 10

3.2 Objectives . . . 10

3.3 Objective fulfilment . . . 10

3.4 Scope and boundaries . . . 11

4 Existing Systems and Gap Analysis 12

4.1 Gap analysis . . . 12

5 System Architecture 13

5.1 Layered architecture . . . 13

5.2 Scan pipeline . . . 14

5.3 Component responsibilities and dependencies . . . 14

5.4 Deployment architecture . . . 15

6 Core Engine and Collection Framework 16

6.1 Input engine and target classification . . . 16

6.2 Scan profiles and module planning . . . 16

6.3 Task scheduling and prioritisation . . . 16

6.4 Worker pool and concurrency . . . 16

6.5 HTTP client, caching, and resilience . . . 17

6.6 Offload pool and configuration . . . 17

7 Collector Modules 18

7.1 Collector inventory . . . 18

7.2 Representative module findings . . . 19

8 Analysis Engines 20

8.1 Correlation engine . . . 20

8.2 Entity model . . . 20

8.3 Risk engine . . . 21

9 Persistence and Reporting 23

9.1 Database design . . . 23

9.2 Migrations . . . 23

9.3 Repositories and the persistence boundary . . . 23

9.4 Reporting subsystem . . . 24

10 Interfaces: CLI, GUI, and Assistant 25

10.1 Command-line interface . . . 25

10.2 Desktop GUI . . . 25

10.3 Visualisations . . . 26

10.4 AI assistant . . . 26

11 Operations: Scheduling, Change Detection, and Cases 27

11.1 Scheduler and change detection . . . 27

11.2 Case management . . . 27

12 Extensibility: Plugin SDK 27

13 Security and Privacy 28

13.1 Threat model . . . 28

13.2 Secrets management . . . 28

13.3 Privacy posture . . . 28

14 Quality Assurance and Test Results 29

14.1 Quality gate . . . 29

14.2 Test strategy . . . 29

14.3 Measured results . . . 29

15 Performance Engineering 30

16 Packaging, Distribution, and CI/CD 31

16.1 Distribution channels . . . 31

16.2 Continuous integration and release . . . 31

17 Development Methodology 31

18 Known Limitations and Roadmap 33

19 Conclusion 33

A Appendix — Reference Data 34

A.1 Runtime dependency versions . . . 34

A.2 Default configuration . . . 34

A.3 Repository layout . . . 34

A.4 Verification and reproduction commands . . . 35

List of Tables

Table 1. Document control summary.
Table 2. Suggested reading paths by audience.
Table 3. Capabilities delivered in release 1.0.0, by dimension.
Table 4. Operational problems motivating the project.
Table 5. Objective fulfilment matrix. All objectives met in release 1.0.0.
Table 6. Representative existing tools and their limitations for a local-first workflow.
Table 7. Feature comparison with common alternatives.
Table 8. Architectural layers and their principal components.
Table 9. Scan pipeline stages, responsibilities, and error handling.
Table 10. Package dependency matrix (abridged).
Table 11. Deployment shapes and their storage isolation.
Table 12. Target types, classification approach, and active collector availability.
Table 13. Engine scan profiles.
Table 14. Task priority levels.
Table 15. Concurrency bounds and their purposes.
Table 16. HTTP resilience mechanisms.
Table 17. Configuration precedence.
Table 18. Built-in collector module inventory (19 modules).
Table 19. Representative findings and fields per module family.
Table 20. Correlation edge types (eight kinds).
Table 21. Entity kinds and their normalisation rules.
Table 22. Risk rule catalog (22 rules). Some rules are predicate-gated on finding data.
Table 23. Category ceilings (left) and rating bands (right). Bands are evaluated highest-first.
Table 24. Database tables.
Table 25. Migration history (v1→v4).
Table 26. Repository surface. The persistence module also records change history by diffing consecutive scans.
Table 27. Report export formats (five). Golden-file tests pin the canonical output to detect drift.
Table 28. CLI command reference. doctor prints a pass/fail environment report.
Table 29. GUI pages (13).
Table 30. Visualisation widgets.
Table 31. Assistant modes.
Table 32. Scheduling and change detection.
Table 33. Case management entities.
Table 34. Plugin SDK mechanisms.
Table 35. Threat summary (details in docs/threat_model.md).
Table 36. Privacy principles and their implementation.
Table 37. Test suite map.
Table 38. Verification results for release v1.0.0.
Table 39. Benchmark baseline (same-host comparability indicators; absolute values vary by machine).
Table 40. Concurrency and performance configuration defaults.
Table 41. Distribution artefacts.
Table 42. CI and release pipeline configuration.
Table 43. Development phase blocks (29 gated phases, all completed).
Table 44. Known limitations and planned enhancements.
Table 45. Pinned runtime dependency versions.
Table 46. Default configuration values.

List of Figures

Figure 1. Layered architecture. Each layer depends only on the layers beneath it; the engine never imports the database, and the UI never contains business logic.
Figure 2. The scan pipeline. Stages are pure where possible; only Execute and Persist perform I/O.
Figure 3. The BaseModule contract implemented by all collectors (built-in and plugin).
Figure 4. Schematic of a correlation graph. Entities are shared across modules; edges are typed and confidence-weighted.
Figure 5. Risk scoring formula. Category ceilings prevent any single category from dominating.
Figure 6. Export pipeline. One model feeds five format exporters; PDF falls back to print HTML when system libraries are absent.
Figure 7. Representative CLI usage.
Figure 8. The mandatory quality gate.
Figure 9. CI/CD flow summary.

1 Executive Summary

IntelXtract is a local-first, account-free desktop platform that automates open-source intelligence (OSINT) work. It accepts a single indicator — a domain, IP address, URL, email address, username, or local file — classifies its type, executes an adaptive plan of specialised asynchronous collectors, correlates the results into an entity-relationship graph, scores the target with a deterministic rule engine, and renders a professional report in five formats. The same engine is exposed through a scriptable command-line interface (CLI) and a PySide6 (Qt 6) desktop GUI that share one persistence layer.

The product addresses the defining problem of OSINT practice: not the scarcity of data, but its fragmentation. Analysts routinely juggle many single-purpose tools, normalise inconsistent outputs by hand, and lose the provenance that makes findings defensible. IntelXtract unifies collection behind a typed module contract, preserves timestamped provenance for every finding, and makes correlation and risk scoring reproducible rather than subjective. All data stays on the analyst's machine: there is no account, no telemetry, and every network egress is explicit and auditable.

19	6	22	5
OSINT collector modules	Supported target types	Deterministic risk rules	Report export formats

756	89.54%	205	v1.0.0
Automated tests passing	Branch coverage	Python source files	Production release

1.1 Delivery summary

Dimension	Delivered capability
Collection	19 collectors across DNS, RDAP/WHOIS, TLS/SSL, certificate transparency, HTTP security headers, technology fingerprinting, favicon hashing, robots.txt, IP geolocation, reverse DNS, DNSBL reputation, email validation and key-gated breach exposure, username presence across 30+ sites, document and image metadata, and news timelines.
Analysis	NetworkX MultiDiGraph correlation with eight typed edge kinds and confidence weights; a 22-rule deterministic risk engine across TLS, Headers, DNS, and Exposure categories with category ceilings, producing a 0–100 score and a critical/high/medium/low rating with per-rule traceability.
Interfaces	Full-parity Typer CLI and a 13-page PySide6 GUI: dashboard, quick/deep scan, results with filters, interactive correlation graph, timeline, world map, history, cases, schedules, plugins, API manager, assistant, and settings.
Operations	SQLite persistence with forward-only migrations v1→v4; recurring scans with cron/interval support, overlap prevention, and change detection; case management with notes, evidence links, and combined reporting.
Extensibility	Manifest-gated Plugin SDK with fail-soft discovery and engine-parity adapters; two reference plugins (Wayback — key-free; VirusTotal — key-gated) and a documented starter template.
Security	Fernet-encrypted API-key vault with OS keyring fallback; log redaction; auto-escaped reports; SSRF redirect guards; an eleven-item threat model and a documented privacy posture.
Quality	756 automated tests passing at 89.54% branch coverage under a mandatory quality gate (ruff, black, mypy, pytest); a fully offline end-to-end integration test; golden-file report drift detection; and performance-budget tests.
Distribution	PyPI wheel and sdist; PyInstaller standalone CLI and GUI binaries; a multi-stage Docker image with a compose stack; SHA-256/512 checksummed GitHub releases; and GitHub Actions CI/CD across Python 3.13 and 3.14.

Table 3. Capabilities delivered in release 1.0.0, by dimension.

1.2 Outcome and assessment

All eleven stated objectives were met. The final release candidate passes the complete quality gate on both supported Python versions, exceeds the 80% branch-coverage floor by more than nine points, and produces byte-stable, reproducible outputs suitable for defensible professional intelligence work. The deliberate exclusions — no active or offensive capability, and no REST API in 1.0.0 — are documented as product decisions with planned mitigations,

rather than unfinished work.

Bottom line

IntelXtract delivers the breadth of a commercial reconnaissance suite — multi-source collection, correlation, risk scoring, case management, scheduling, and professional reporting — as a private, local-first application with no account, no telemetry, and no recurring cost, verified to a high standard of automated testing.

2 Introduction and Background

2.1 What open-source intelligence is

Open-source intelligence (OSINT) is the discipline of collecting and analysing information from publicly available sources to produce actionable intelligence. In modern security practice it underpins attack-surface discovery, brand-protection monitoring, due diligence, incident triage, and threat-actor profiling. The raw material is abundant — DNS records, WHOIS/RDAP registration data, TLS certificates, HTTP response headers, breach corpora, and public news feeds — but it is scattered across dozens of incompatible services, each with its own interface, rate limits, and output schema.

The name IntelXtract encodes the product's purpose: **Intel**ligence e**Xtract**ion. The application consolidates the reconnaissance workflow into one deterministic pipeline that a single analyst can drive interactively or script for automation.

2.2 The OSINT landscape and its friction

Practitioners routinely combine tools such as dig, whois, openssl s_client, curl, theHarvester, certificate-transparency front-ends, and vendor APIs. Each answers one narrow question: what are the A records; who registered the domain; when does the certificate expire; which security headers are set? The analytical value, however, lies in the connections between these answers. A certificate's subject-alternative-name field may reveal sibling domains; an MX record may point to a provider that implies breach exposure; a favicon hash may link unrelated hosts to the same infrastructure.

Manual correlation is slow, error-prone, and difficult to reproduce. Analysts re-run the same commands, copy results into spreadsheets, and lose provenance. Existing automation platforms tend to be cloud-hosted (raising data-sovereignty concerns), closed-source, or expensive per-query. IntelXtract was built to close that gap.

2.3 Motivation

1. **Fragmentation is the bottleneck, not collection.** Individual collectors are simple; orchestrating them, normalising their output, and preserving provenance is the hard part.
2. **Analysts need reproducible evidence.** A finding that cannot be traced to a timestamped source is not intelligence. Determinism and auditability are therefore first-class requirements.
3. **Privacy is non-negotiable.** Reconnaissance data is sensitive; a tool that silently uploads targets to a third party is unacceptable for many professional contexts.

These observations shaped four guiding principles: unify collection behind a typed module contract; correlate and score deterministically; persist everything locally with full provenance; and make every network egress explicit.

3 Problem Statement and Objectives

3.1 Problem statement

The absence of a unified, local, reproducible OSINT workbench creates concrete operational problems:

Problem	Consequence for the analyst
Inefficiency	A single-domain investigation may require five to ten tools and manual merging; this does not scale to cases covering dozens of indicators.
Inconsistency	The same fact (e.g., a certificate issuer) is represented differently across tools, defeating comparison across targets and scans.
Loss of provenance	Ad-hoc collection leaves no reliable record of when a datum was observed or which source produced it, undermining defensibility.
No correlation	Isolated facts are rarely actionable; relationships between domains, IPs, certificates, emails, and identities stay invisible.
No prioritisation	Flat finding lists give no guidance on what matters; analysts need a transparent, defensible risk signal.
Privacy exposure	Cloud-hosted tools exfiltrate target data by design — unacceptable for sensitive investigations.
Poor reproducibility	Manual steps cannot be re-run identically, so change detection is impractical.

Table 4. Operational problems motivating the project.

> **Problem statement**
> There is a need for a privacy-preserving, local-first, extensible desktop platform that automates the collection of open-source intelligence across multiple indicator types, correlates findings into an explainable relationship graph, scores risk deterministically, and produces reproducible professional reports — all while keeping the analyst in full control of data egress.

3.2 Objectives

Eleven measurable objectives were defined at inception and used as the acceptance criteria for the release:

1. **Multi-type collection** — support Domain, IP, URL, Email, Username, and File targets with specialised passive collectors.
2. **Unified orchestration** — a single asynchronous engine that plans, schedules, executes, and aggregates module runs with deterministic ordering.
3. **Correlation** — build an entity-relationship graph linking findings across modules with typed edges and confidence scoring.
4. **Explainable risk scoring** — a deterministic rule engine with documented weights, category ceilings, and an auditable rule inventory.
5. **Dual interface** — a fully scriptable CLI and an equivalent GUI sharing one engine.
6. **Local persistence with provenance** — versioned SQLite storage with forward-only migrations and typed repositories.
7. **Reproducible reporting** — byte-consistent exports in at least five formats.
8. **Extensibility** — a documented plugin SDK so third parties can add collectors without modifying core code.
9. **Security and privacy** — encrypt keys at rest, disable AI/telemetry by default, and document the threat model.
10. **Quality** — at least 80% branch coverage with a passing quality gate and fully offline integration tests.
11. **Packaged release** — package via PyInstaller, Docker, and PyPI, driven by CI/CD.

3.3 Objective fulfilment

#	Objective	Delivered in	Evidence
1	Multi-type passive collection (6 active types)	Module layer	19 collectors; catalog in Section 6
2	Unified async orchestration	Core engine	Scheduler + bounded worker pool (Section 5)
3	Entity-relationship correlation graph	Correlation engine	8 edge kinds; deterministic MultiDiGraph (Section 7)
4	Explainable deterministic risk scoring	Risk engine	22 rules, ceilings, rating bands (Section 8)
5	Dual CLI and GUI parity	Interfaces	Typer CLI + 13-page PySide6 GUI (Section 10)
6	Local persistence with provenance	Data layer	SQLite; migrations v1→v4; repositories (Section 9)
7	Reproducible 5-format reporting	Reporting	One model, five exporters (Section 9.4)
8	Third-party extensibility	Plugin SDK	Manifest-gated loader; reference plugins (Section 12)
9	Security and privacy by design	Vault + docs	Fernet vault; threat model; privacy (Section 13)
10	Quality ≥ 80% branch coverage	Test suite	756 tests, 89.54% coverage (Section 14)
11	Packaged multi-channel release	Release eng.	Wheel/sdist, binaries, Docker, CI/CD (Section 16)

Table 5. Objective fulfilment matrix. All objectives met in release 1.0.0.

3.4 Scope and boundaries

In scope

- Passive reconnaissance against publicly available data sources: DNS-over-HTTPS, RDAP/WHOIS, certificate transparency, HTTP(S) response inspection, key-gated breach APIs, public news RSS, and local file metadata parsing.
- Six active target types with collectors plus two additional classified types (Hash, File metadata), a pluggable module framework, deterministic correlation and risk scoring, five-format reporting, and scan scheduling with change detection.
- Local desktop usage on Windows, Linux, and macOS, plus headless CLI and Docker execution.
- Case management for grouping scans, notes, and evidence; and an optional, off-by-default AI assistant that summarises already-collected evidence.

Out of scope

- Active exploitation or intrusive scanning. The tool performs no exploitation, brute-forcing, or vulnerability probing beyond reading public metadata — a deliberate ethical boundary.
- Authenticated or credentialed access to third-party systems.
- Real-time streaming or threat-feed subscription management.
- Multi-user server deployment. The architecture is single-user desktop-first; the Docker image provides headless execution, not a multi-tenant service.
- Autonomous AI decision-making. The assistant interprets evidence but never triggers scans or changes data on its own.

Assumptions

- The host has outbound access to configured public sources, or the user operates in an offline/cached mode.
- The analyst is authorised to investigate the targets and complies with applicable law and terms of service.
- Python 3.13 or newer is available for source installs; frozen binaries bundle their own runtime.

4 Existing Systems and Gap Analysis

OSINT tooling spans a spectrum from single-purpose command-line utilities to commercial link-analysis platforms. IntelXtract was scoped against representative members of that spectrum.

Tool	Category	Strengths	Limitations relative to IntelXtract
theHarvester	CLI reconnaissance	Broad source coverage; mature	No correlation graph, no risk scoring, no GUI, no persistence or reporting model
Maltego	Graph analysis	Excellent link analysis; transforms	Commercial; GUI-centric; transforms often cloud-hosted; not local-first
SpiderFoot	Automation platform	Many modules; correlator; web UI	Server-oriented; heavier deployment; weaker local-desktop story
Recon-ng	CLI framework	Scriptable; modular	Steep learning curve; no first-class risk scoring or report templates
dig / whois / openssl	Point tools	Ubiquitous; precise	No aggregation, persistence, correlation, or automation
Shodan / Censys	Data services	Rich internet-wide data	Proprietary; per-query cost; data leaves the analyst's machine

Table 6. Representative existing tools and their limitations for a local-first workflow.

4.1 Gap analysis

Across the surveyed tools, three gaps recur. First, aggregation is usually manual: each tool answers a narrow question and the analyst performs correlation by hand. Second, prioritisation is absent or opaque: few tools provide a transparent, reproducible risk signal. Third, privacy and sovereignty are secondary: the most capable platforms are cloud-hosted and account-gated. IntelXtract targets exactly this intersection — automated aggregation, deterministic prioritisation, and local-first privacy — as a single cohesive product.

Capability	IntelXtract	dig/whois	theHarvester	Maltego	Shodan
Dual CLI + desktop GUI	Yes	No	No	Desktop	Web
Local-first, no account	Yes	Yes	Yes	Partial	No
Correlation graph	Yes	No	No	Yes	No
Deterministic risk score	Yes (22 rules)	No	No	Partial	No
Multi-format reports	5 formats	No	Text/HTML	Yes	Yes
Email / breach checks	Yes (key-gated)	No	Partial	Transforms	No
Plugin SDK	Yes	No	No	Transforms	No
Encrypted key vault	Yes	N/A	No	Yes	N/A
Case management	Yes	No	No	Yes	No
Recurring scans + diff	Yes	No	No	Partial	Partial
Open source	GPL-3.0+	Yes	Yes	No	No

Table 7. Feature comparison with common alternatives.

5 System Architecture

IntelXtract is organised as a small number of cooperating subsystems with strict separation of concerns. The scan engine is pure Python and fully decoupled from both the persistence layer and the user interfaces; modules receive a typed context object rather than direct access to infrastructure. This keeps the system testable without a database or network and allows the CLI and GUI to drive the identical engine.

5.1 Layered architecture



Figure 1. Layered architecture. Each layer depends only on the layers beneath it; the engine never imports the database, and the UI never contains business logic.

Layer / subsystem	Responsibility	Key components
Interfaces	User interaction with full functional parity	cli/ (Typer), gui/ (PySide6), shared AppContext
Core engine	Plan, schedule, execute, and aggregate scans	ScanEngine, InputEngine, TaskScheduler, WorkerPool, OffloadPool, AsyncTTLCache, HttpClient
Module layer	Passive collection behind one contract	BaseModule subclasses; ModuleContext; 19 built-in modules
Analysis	Correlation and risk	correlation.build_graph; risk.assess_risk (pure function)
Persistence	Durable local state with provenance	aiosqlite; repositories; migrations v1→v4; persistence sink
Reporting	Reproducible multi-format exports	ReportModel builder; JSON/CSV/Markdown exporters; Jinja2 HTML; WeasyPrint PDF
Operations	Automation and investigation workflow	Scheduler (interval/cron); change detection; case management; audit logging
Security	Secrets and guardrails	Fernet vault; keyring fallback; log redaction; SSRF guards; threat model
Extensibility	Third-party collectors	Plugin SDK; manifest validation; fail-soft loader; engine-parity adapter
Assistant	Grounded narrative over evidence	Disabled-by-default provider facade; deterministic offline path; OpenAI-compatible remote path

Table 8. Architectural layers and their principal components.

5.2 Scan pipeline

ScanEngine.scan() implements a fixed, ordered pipeline. Each stage is independently tested and the sequence is identical for quick, deep, custom, and plugin-augmented runs, which is what makes the engine's behaviour predictable and reproducible.

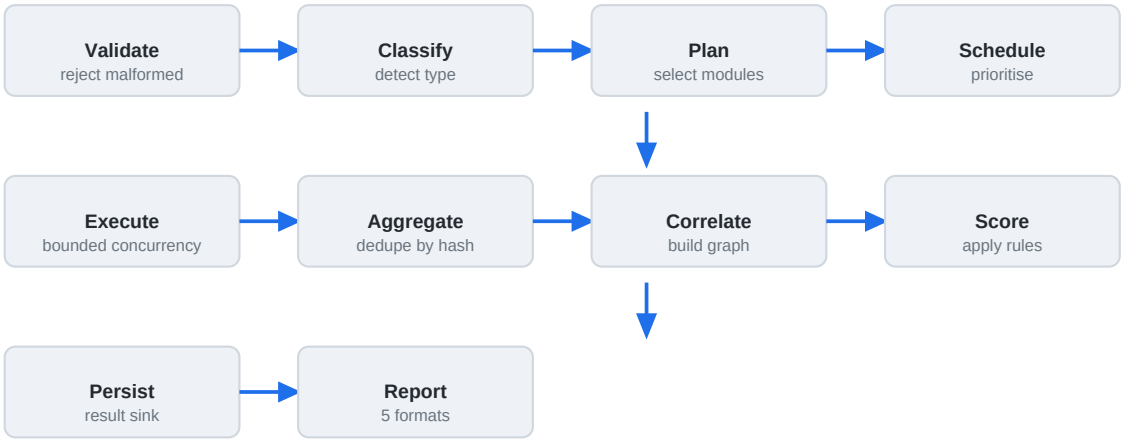


Figure 2. The scan pipeline. Stages are pure where possible; only Execute and Persist perform I/O.

Stage	Behaviour	Key error handling
Validate	Rejects malformed indicators with a clear error before any network activity.	Raises a typed validation error
Classify	Maps the raw string to a TargetType (Domain, IP, URL, Email, Username, Hash, File, Unknown).	Unknown types surface a helpful message
Plan	Selects modules by scan profile; modules whose required keys are unset are skipped gracefully.	Missing keys degrade coverage, never fail the scan
Schedule	Creates a prioritised Task per selected module in the TaskScheduler.	Deterministic ordering by priority then name
Execute	WorkerPool runs tasks with bounded concurrency and per-task timeouts.	Timeouts recorded as TIMEOUT, not crashes
Aggregate	Normalises findings and deduplicates by a stable SHA-256 content hash.	Unknown exception types recorded per module
Correlate	Extracts entities and adds typed, confidence-weighted edges.	No entities yields no graph
Score	Applies the rule catalog and category ceilings to produce a score and rating.	Pure function; no I/O
Persist	An injected sink stores target, scan, findings, and run metadata.	Engine never imports the database

Table 9. Scan pipeline stages, responsibilities, and error handling.

5.3 Component responsibilities and dependencies

The package dependency graph is deliberately acyclic. Lower-level packages (core primitives, database, modules) never import higher-level ones (engine, interfaces). The dependency direction is: interfaces → engine → modules → database/core-primitives, with reporting depending only on the shared model and analysis primitives.

Package	Depends on	Consumed by
core (primitives, config, cache, http, entities)	—	everything
database (models, repos, migrations, persistence)	core	engine sinks, interfaces
modules (19 collectors)	core	engine
core.engine / scheduler / worker_pool	core, modules	interfaces
core.correlation / core.risk	core.entities	engine, reporting
reports	core (models), analysis	interfaces
cli / gui	engine, database, reports, assistant, plugins	— (entry points)
plugins	core (context), modules contract	engine via adapter

Table 10. Package dependency matrix (abridged).

5.4 Deployment architecture

IntelXtract runs in three deployment shapes from one codebase. The desktop build is the primary experience. The Docker image provides a reproducible headless runtime for servers and CI. The compose stack adds a GUI service (for display-forwarded hosts) and a long-running scheduler service, with named volumes partitioning data, configuration, and plugins.

Deployment shape	Entry point	Isolation / storage
Desktop (Windows/Linux/macOS)	intelxtract-gui (PyInstaller or source)	Local config/, logs/, cache/, exports/ and SQLite file
Headless CLI	intelxtract scan|report|schedule|...	Same local paths; scriptable and CI-friendly
Docker (headless)	intelxtract (runtime image, ~150 MB)	Named volume intelxtract-data for DB, exports, logs, cache
Docker compose	intelxtract, intelxtract-gui, intelxtract-scheduler	Named volumes for data and config; plugins and templates mounted read-only

Table 11. Deployment shapes and their storage isolation.

6 Core Engine and Collection Framework

6.1 Input engine and target classification

The input engine is the entry point of every scan. It classifies a raw indicator into one of eight target types and normalises it (lowercasing domains, stripping wildcard prefixes, canonicalising IPs, and extracting hosts and ports from URLs). Classification is deterministic and covered by dedicated unit tests.

Target type	Recognition approach	Active collector?
Domain	Contains a dot, no scheme, valid hostname labels	Yes (7 modules)
IP	Parses as IPv4/IPv6 via ipaddress	Yes (3 modules)
URL	Has a scheme (http/https)	Yes (5 modules)
Email	Contains @ with valid local and domain parts	Yes (2 modules)
Username	Plain identifier, no dot/space	Yes (1 module)
File	Existing path to a local file	Yes (metadata module)
Hash	Recognised hash shape (MD5/SHA-1/SHA-256)	No collector yet (classified only)
Unknown	None of the above	Rejected with guidance

Table 12. Target types, classification approach, and active collector availability.

6.2 Scan profiles and module planning

A scan profile selects which modules run. The quick profile targets the highest-signal checks for fast triage; the deep profile runs the full plan; the custom profile runs an explicit module list. Planning also filters by target type and by key availability, so a scan never fails because an optional API key is missing — the module is simply skipped and reported as such.

Profile	Intent	Typical modules
Quick	Fast triage with the highest-signal checks	dns, http, ssl, whois (and target-type equivalents)
Deep	Comprehensive coverage	All applicable modules for the target type
Custom	Explicit, repeatable module selection	Exactly the modules the user names via --modules

Table 13. Engine scan profiles.

6.3 Task scheduling and prioritisation

Each selected module becomes a Task with a priority. The TaskScheduler orders tasks by priority and then by a stable secondary key, which preserves deterministic execution order and therefore deterministic output. Higher-priority modules (those most likely to yield correlatable entities) are scheduled first, improving the usefulness of partial results if a deep scan is interrupted.

Priority	Representative modules	Rationale
High	dns, whois, ssl, certificate	Foundation of entity extraction and correlation
Medium	http, headers, tech, subdomain, rdap-derived	Enrichment and fingerprinting
Low	news, favicon, reputation, geo, metadata	Contextual and third-party enrichment

Table 14. Task priority levels.

6.4 Worker pool and concurrency

The worker pool executes tasks with bounded concurrency. Each task is wrapped in asyncio.wait_for with a per-task timeout, so a slow or hostile target cannot stall a scan. A sentinel-based shutdown drains the queue cleanly, and a completion hook feeds progress to the CLI and GUI without coupling the engine to either. Concurrency is bounded end-to-end — there are no unbounded queues, threads, or caches in the hot path.

Mechanism	Bound	Purpose
Worker concurrency	scan.max_workers (default 8)	Limits simultaneous module runs
Per-task timeout	scan.default_timeout (default 30 s)	Prevents a single slow target from stalling the scan
Offload workers	performance.offload_workers (default 4)	Runs blocking parsing off the event loop
Global HTTP in-flight	network.max_concurrency (default 16)	Caps total concurrent requests
Per-host HTTP in-flight	network.max_per_host (default 4)	Prevents overwhelming any single host

Table 15. Concurrency bounds and their purposes.

6.5 HTTP client, caching, and resilience

The HTTP client owns one keep-alive aiohttp.TCPConnector per client with total and per-host connection caps and a DNS cache. Requests acquire a global slot, then a per-host slot, then pass through optional rate limiters — an ordering chosen to be deadlock-free. A shared AsyncTTLCache provides single-flight semantics, and HTTP responses carrying ETag or Last-Modified are revalidated with conditional requests, serving cached bodies on 304 responses. Retries use exponential backoff with equal jitter and are restricted to idempotent methods.

Concern	Design	Benefit
Connection reuse	One keep-alive connector per client with DNS caching	Lower latency and fewer TLS handshakes
Conditional requests	ETag / Last-Modified revalidation; cached 304 bodies	Reduced bandwidth and polite behaviour
Single-flight cache	AsyncTTLCache coalesces concurrent misses	Avoids duplicate upstream work
Backoff	Exponential with equal jitter; idempotent methods only	Resilient without inducing retry storms
Admission control	Global then per-host semaphores, then rate limiters	Bounded, deadlock-free load

Table 16. HTTP resilience mechanisms.

6.6 Offload pool and configuration

Genuinely blocking work — PDF, EXIF, and OOXML parsing, and the username registry load — runs on a bounded offload thread pool so the asyncio event loop is never stalled. Configuration is layered and validated, with a clear precedence: environment variables override the encrypted vault, which overrides settings.json, which overrides built-in defaults.

Configuration source	Precedence	Notes
Environment variables (INTELXTRACT_ prefix)	1 (highest)	Nested keys use __ as separator; .env loaded on startup
Encrypted API-key vault	2	Fernet ciphertext in the database; never written to settings.json
settings.json / database settings	3	User-editable runtime settings (no secrets)
Built-in defaults	4 (lowest)	Documented in Appendix B

Table 17. Configuration precedence.

7 Collector Modules

Every collector subclasses BaseModule and declares three pieces of metadata: **name**, the **target types** it supports, and any **API keys** it requires. The planner skips modules whose keys are unconfigured and modules whose target types do not include the classified type. Collectors stay thin because shared conveniences — HTTP helpers, a retry decorator, and a timeout wrapper — live in the base layer, and a common parser normalises findings and computes a stable content hash for deduplication.

```
class BaseModule(ABC):
    name: str = ""
    target_types: tuple[TargetType, ...] = ()
    requires_keys: tuple[str, ...] = ()
    description: str = ""

    def validate(self, target: str) -> bool: ...

    @abstractmethod
    async def run(self, target: str, ctx: ModuleContext) -> ModuleResult: ...
```

Figure 3. The BaseModule contract implemented by all collectors (built-in and plugin).

7.1 Collector inventory

#	Module	Target	Key	Collects
1	dns	Domain	—	A/AAAA/CNAME/NS/MX/TXT over DNS-over-HTTPS; SPF, DMARC, and DKIM selector probing
2	http	Domain	—	HTTPS status, redirect chain, robots/sitemap presence, security-header inventory
3	ssl	Domain	—	Leaf certificate metadata: subject, issuer, validity, SANs; expiry and self-signed flags
4	whois	Domain	—	RDAP registrar, registration dates, nameservers, status flags, DNSSEC, contacts
5	subdomain	Domain	—	Passive subdomain discovery via certificate transparency (crt.sh)
6	certificate	Domain	—	Certificate chain, live TLS probe, CT history, related names
7	news	Domain	—	News timeline from Google News RSS
8	geo	IP	—	Geolocation and network facts (ASN, ISP, organisation), cached 24 h
9	rdns	IP	—	PTR lookup with forward-confirmation to resist spoofing
10	reputation	IP	AbuseIPDB (opt.)	Spamhaus ZEN DNSBL listing; optional AbuseIPDB enrichment
11	favicon	URL	—	Favicon fingerprint (FNV-1a over base64 MD5) for infrastructure clustering
12	headers	URL	—	Response headers, cookie flags, server disclosure, security-header gaps
13	http_methods	URL	—	Allowed methods (OPTIONS/TRACE), page title, content type
14	robots	URL	—	robots.txt groups and sitemap URL extraction
15	tech	URL	—	Technology-stack fingerprinting
16	email	Email	—	Validation, MX/SPF/DMARC on the mail domain, Gravatar profile
17	breach	Email	HIBP	Breach exposure lookup (key-gated; skipped when unset)
18	username	Username	GitHub (opt.)	Presence across 30+ sites plus profile enrichment
19	metadata	File	—	PDF properties, image EXIF (including GPS), OOXML office core properties

Table 18. Built-in collector module inventory (19 modules).

7.2 Representative module findings

Module	Representative findings	Notable data fields
dns	A/AAAA/CNAME/NS/MX/TXT records; SPF present/absent; DMARC present/absent; DKIM selectors	type, records, policy, all
http	HTTPS responds; redirect chain; HTTPS unreachable; labelled header present/unreachable; missing header	status, chain, robots, sitemap
ssl	Certificate; subject alternative names; self-signed; expired	subject_cn, issuer, subject_alt_names, not_after
whois	Registrar; registration dates; nameservers; status flags; DNSSEC signed/not signed; role contacts	registrar, nameservers, email, org
headers	Header present/missing; server disclosure; insecure cookie; response profile	header, value, flags
username	Presence across sites; profile enrichment	site, url, found
metadata	PDF properties; image EXIF (including GPS); OOXML core properties	author, created, gps, software
news	Timeline of recent articles; none found	articles[] (title, url, published)

Table 19. Representative findings and fields per module family.

> **Correctness details worth noting**
>
> Reverse DNS is forward-confirmed: a PTR hostname is resolved back to an address and only reported on a round-trip match. Findings are deduplicated by a stable content hash. Blocking parsers run on the offload pool. The ssl module's expiry and self-signed detections feed the two heaviest TLS risk rules.

8 Analysis Engines

8.1 Correlation engine

The correlation engine converts a flat finding list into a connected entity graph using a NetworkX MultiDiGraph. Node identity is derived from a normalised (type, value) pair hashed to a stable key (kind:<12-hex>); edges carry a semantic type and a confidence weight. When the same edge is derived twice, the maximum confidence is kept. Nodes and edges are sorted by stable keys, so exports, persistence, and diffs are byte-stable.

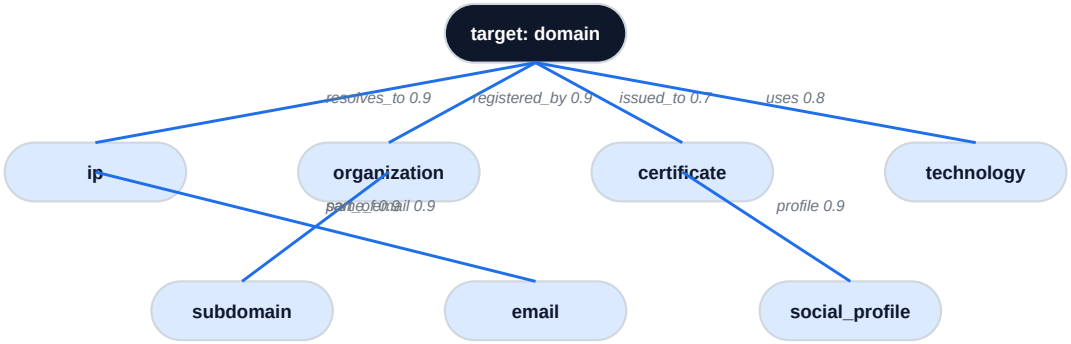


Figure 4. Schematic of a correlation graph. Entities are shared across modules; edges are typed and confidence-weighted.

Edge type	Meaning	Typical confidence	Example
resolves_to	domain → IP / alias	0.4–0.9	A/AAAA or CNAME resolution
registered_by	domain → registrar organisation	0.9	RDAP registration data
issued_to	domain / certificate → issuer	0.7–0.9	TLS certificate chain
same_email	email → domain / username	0.7–0.9	Shared contact identity
part_of	base domain → subdomain	0.9	Certificate-transparency subdomains
mentions	publisher → target	0.4	News article linkage
profile	username → social profile	0.9	Presence checks
uses	domain → technology	0.8	Technology fingerprinting

Table 20. Correlation edge types (eight kinds).

8.2 Entity model

Entities are the lowest common denominator across modules: an IP surfaced by both a DNS A record and a certificate entry becomes a single node. Each module declares extractors against finding data; the EntityBag aggregates occurrences into one entity with a count and a set of contributing sources, so the graph reflects agreement across independent collectors.

Entity kind	Normalisation
domain	lowercase; strip trailing dot and wildcard prefix
subdomain	lowercase; strip leading wildcard
email	lowercase
username	strip leading @; lowercase
ip	canonical ipaddress form
organization	whitespace-collapsed
certificate	whitespace-collapsed; uppercase serial/identity
document	lowercase filename
social_profile	URL verbatim
technology	lowercase

Table 21. Entity kinds and their normalisation rules.

8.3 Risk engine

The risk engine is a pure function: identical findings always yield an identical score. It walks every finding, applies the documented rule catalog, de-duplicates identical (rule, finding) pairs, computes per-category sub-scores with ceilings, and caps the total at 100. Each matched rule is recorded with its id, category, label, points, and the finding it fired on, so the score is fully traceable and explainable.

```
score = min(100.0, sum(min(category_score, ceiling[category])
                       for category in categories))
```

Figure 5. Risk scoring formula. Category ceilings prevent any single category from dominating.

Rule ID	Category	Wt	Trigger
tls_cert_expired	TLS	35	Certificate expired
tls_self_signed	TLS	18	Self-signed certificate
tls_no_https	TLS	10	HTTPS unreachable
headers_trace	HEADERS	15	TRACE method enabled
headers_hsts	HEADERS	9	Strict-Transport-Security missing
headers_csp	HEADERS	9	Content-Security-Policy missing
headers_cookie_no_secure	HEADERS	7	Cookie set without Secure flag
headers_xframe	HEADERS	4	X-Frame-Options missing
headers_server	HEADERS	4	Server banner disclosed
headers_xcto	HEADERS	3	X-Content-Type-Options missing
headers_referrer	HEADERS	2	Referrer-Policy missing
headers_cookie_weak	HEADERS	2	Cookie missing HttpOnly/SameSite
headers_permissions	HEADERS	2	Permissions-Policy missing
dns_no_dmarc	DNS	6	No DMARC record
dns_no_spf	DNS	4	No SPF record
dns_dnssec_absent	DNS	3	Domain not DNSSEC-signed
dns_no_mx	DNS	5	Domain accepts no mail (no MX)
dns_spf_allow_all	DNS	6	SPF policy exactly +all
dns_dmarc_none	DNS	6	DMARC policy p=none
exposure_breach	EXPOSURE	20	Address found in breach corpus
exposure_gps	EXPOSURE	10	GPS coordinates in document metadata
exposure_disposable	EXPOSURE	4	Disposable mail provider

Table 22. Risk rule catalog (22 rules). Some rules are predicate-gated on finding data.

Category	Rules	Ceiling	Rating	Threshold	Verdict
TLS	3	40	Critical	≥ 75	Immediate remediation required
HEADERS	10	30	High	≥ 50	Significant weaknesses found
DNS	6	20	Medium	≥ 25	Moderate weaknesses found
EXPOSURE	3	25	Low	≥ 0	Low overall exposure

Table 23. Category ceilings (left) and rating bands (right). Bands are evaluated highest-first.

9 Persistence and Reporting

9.1 Database design

All state lives in a single SQLite database accessed asynchronously through aiosqlite. The schema is normalised around targets → scans → findings, with satellite tables for reports, schedules, cases, plugins, keys, settings, logs, and per-target history. Referential integrity uses CASCADE for scan-owned rows and SET NULL for soft links, and indices cover every foreign key and the common query filters.

Table	Purpose	Key columns
targets	Classified indicators	id, value, type (unique value+type)
scans	Scan runs	id, target_id FK, uuid, mode, status, runs_json
findings	Normalised intelligence	id, scan_id FK, module, title, severity, confidence, data, evidence, content_hash
reports	Generated report artefacts	id, scan_id FK, path, format
history	Per-target change events	target_id, scan_id, event_type, payload
plugins	Plugin metadata and state	name, version, api_version, enabled, source, error
api_keys	Encrypted secrets	provider (unique), ciphertext, hint
settings	Runtime settings	key (PK), value (JSON), updated_at
schedules	Recurring scans	target, mode, interval_seconds/cron, enabled, status, next_run_at
cases	Investigations	name (unique), description, status
case_notes	Investigation notes	case_id FK, body
case_evidence	Evidence links	case_id, scan_id, finding_id, kind, label
logs	Redacted audit events	level, message, source, context
schema_version	Applied migrations	version (PK), name, applied_at

Table 24. Database tables.

9.2 Migrations

The schema evolves through forward-only, immutable migrations recorded in schema_version. The runner validates that versions are unique and contiguous from 1, applies only pending migrations in order, and records the failure point without marking a failed migration as applied. The initial SQL is loaded with importlib.resources so it resolves correctly inside a PyInstaller bundle.

Version	Name	Change
1	initial schema	targets, scans, findings, reports, plugins, api_keys, logs, history, settings
2	finding normalized fields	add findings.title, evidence, content_hash, collected_at
3	recurring schedules	add schedules table + indices
4	case management	add cases, case_notes, case_evidence

Table 25. Migration history (v1→v4).

9.3 Repositories and the persistence boundary

All SQL is confined to the repository and migration modules; rows surface to the rest of the application as frozen, slotted dataclasses with JSON columns parsed on read and serialised on write. Crucially, the engine never imports the database. Instead, persistence is injected as an async result-sink callable. This boundary is what makes the engine unit-testable without a database and keeps the UI free of SQL.

Repository	Principal operations
TargetRepository	get_or_create, get, list
ScanRepository	create, create_for_result, list, list_detailed, latest_for_target, stats
FindingRepository	bulk_create, list_for_scan, count_for_scan, get
ScheduleRepository	implements the ScheduleStore protocol consumed by the runner
CaseRepository	cases plus notes and evidence
Report / Plugin / ApiKey / Log / History / Setting	typed accessors for their tables

Table 26. Repository surface. The persistence module also records change history by diffing consecutive scans.

9.4 Reporting subsystem

All exporters consume one shared ReportModel, so every format remains mutually consistent. The builder re-derives correlation and risk when they are not supplied, allowing reports to render correctly even for scans persisted before those subsystems existed. The model includes an executive summary (top three findings by severity), evidence references, methodology, and a merged timeline.

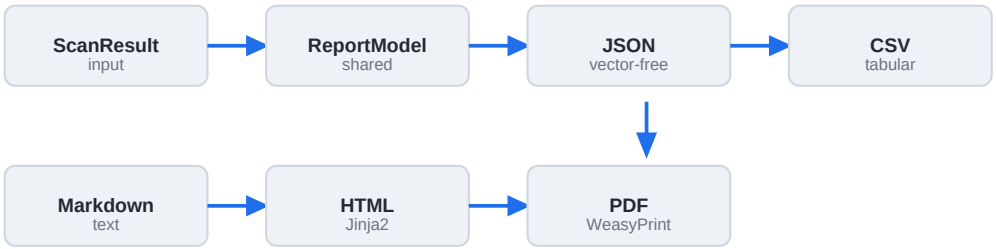


Figure 6. Export pipeline. One model feeds five format exporters; PDF falls back to print HTML when system libraries are absent.

Format	Producer	Structure / notes
JSON	Canonical model serialisation	Full fidelity, machine-consumable, 2-space indent
CSV	Row per finding	Severity, module, title, confidence, evidence (tabular-truncated)
Markdown	Structured text builder	Headings, tables, timeline, methodology; escaped cells
HTML	Jinja2 (report.html.j2)	Branded, self-contained, auto-escaped, print-optimised
PDF	WeasyPrint (HTML to PDF)	Falls back to print-friendly HTML with a warning if system libraries are absent

Table 27. Report export formats (five). Golden-file tests pin the canonical output to detect drift.

10 Interfaces: CLI, GUI, and Assistant

10.1 Command-line interface

The CLI is built on Typer with Rich rendering and provides full functional parity with the GUI for headless and automation use. The root command is intelxtract; six sub-applications group related commands. The root callback configures logging (with -v switching to DEBUG) and supports --version.

Command	Purpose	Key options
scan <target>	Run a scan and persist it	--mode/-m, --modules/-M, --format/-f console|json, --db
report <scan_id>	Render a saved scan	--format json|html|pdf|csv|md, --out, --db
explain <scan_id>	Rule-based / AI narrative	--kind summary|next-steps|finding|compare, --json
history list|show	Browse past scans	--target/-t, --status/-s
config get|set|list|path	Inspect / modify configuration	dotted keys (e.g., cache.ttl)
key list|set|get|delete	Manage encrypted API keys	--value, --reveal, --yes
plugin list|info|enable|disable	Manage plugins	--db
schedule add|list|remove|run	Recurring scans	--every, --cron, --mode, --once, --poll
case create|list|show|note|attach|export	Investigations	--description, --status
doctor	Environment health check	—
update	Check PyPI for a newer version	—

Table 28. CLI command reference. doctor prints a pass/fail environment report.

```
intelxtract scan example.com --mode deep
intelxtract report 1 --format html --out exports/report.html
intelxtract report 1 --format pdf  --out exports/report.pdf
intelxtract schedule add example.com --every 24h --mode quick
intelxtract case create "Q4 vendor review" --description "Due diligence"
intelxtract doctor
```

Figure 7. Representative CLI usage.

10.2 Desktop GUI

The PySide6 desktop client drives the same engine as the CLI on a background QThread running an asyncio event loop. Engine callbacks are marshalled to the UI thread via Qt signals, so no widget is ever touched from a worker thread and the interface stays responsive during scans. The GUI covers 13 pages and a set of custom-painted visualisations.

Page	Function
Dashboard	Recent scans, risk summary, quick actions
Quick Scan / Deep Scan	One-click profile scan; full profile with module selection
Results	Findings table with severity filters and detail pane
Visualisations	Interactive correlation graph, severity/module charts, timeline, world map
History	Browse and reopen past scans; change events
Reports	Generate and export reports in any supported format
Cases	Investigations with notes, evidence links, and combined export
Schedules	Manage recurring scans; run the scheduler
Plugins	Enable/disable plugins; view manifest metadata
API Manager	Manage encrypted API keys (masked)
Assistant	Opt-in AI narrative and explanations
Settings / About	Configuration values; version, licence, and project links

Table 29. GUI pages (13).

10.3 Visualisations

Visualisation widgets are painted directly with Qt, so the product carries no external plotting dependency. The correlation graph lays out the MultiDiGraph, the timeline orders events by timestamp, and the world map plots IP geolocation points on a bundled equirectangular base map.

Widget	Module	Description
Severity / module charts	viz/charts.py	Bar and donut summaries of findings by severity and module
Correlation graph	viz/graph_widget.py	Interactive entity-graph rendering
Timeline	viz/timeline.py	Chronological scan and news events
World map	viz/world_map.py	Geographic plotting of IP geolocation findings

Table 30. Visualisation widgets.

10.4 AI assistant

The AI assistant is disabled by default and strictly opt-in. It has two layers: a deterministic, offline analysis path that always works, and an optional OpenAI-compatible chat path that activates only when the user configures an endpoint. Assistant answers are labelled AI-generated, and the remote path is treated in the threat model as an explicit, disclosed egress.

Mode	Provider	Network	Description
Off (default)	DisabledAssistant	None	Returns a disabled warning; no data leaves the host
Deterministic	rule-based Assistant	None	Summary, next-steps, finding, and comparison narratives
Remote	OpenAI-compatible	HTTPS to user endpoint	Sends the report payload to the user's /chat/completions endpoint

Table 31. Assistant modes.

11 Operations: Scheduling, Change Detection, and Cases

11.1 Scheduler and change detection

Recurring scans can be configured with a fixed interval or a five-field cron expression. The scheduler enforces overlap prevention (a target is never scanned concurrently with itself) and records a next-run time. Between successive scans of the same target, the persistence layer diffs findings and records discrete change events, which is what makes 'what changed since yesterday?' a first-class, reproducible query rather than a manual chore.

Concern	Design	Benefit
Interval scheduling	Fixed seconds between runs	Simple recurring checks
Cron scheduling	Five-field cron expressions	Calendar-aware schedules
Overlap prevention	Per-target in-flight guard	No duplicate or conflicting runs
Change detection	Diff of consecutive scans recorded as events	Reproducible drift history

Table 32. Scheduling and change detection.

11.2 Case management

Cases group scans, notes, and evidence into an investigation. A case can reference any number of scans and specific findings, carry free-text notes, and be exported as a combined report that de-duplicates findings across its scans. Cases give multi-indicator investigations a durable, reportable structure.

Entity	Contents	Relationship
Case	Name (unique), description, status	Root of an investigation
Case note	Free-text body	Many per case
Case evidence	Link to a scan or a specific finding, with kind and label	Many per case

Table 33. Case management entities.

12 Extensibility: Plugin SDK

Third-party collectors integrate without modifying core code. The loader discovers plugin directories, validates a declarative manifest, and wraps each plugin in an engine-parity adapter so plugins are planned, executed, timed out, and reported exactly like built-in modules. Discovery is fail-soft: a broken plugin is recorded with an error and disabled — it can never take down a scan or the application.

Aspect	Mechanism
Contract	PluginBase with the same typed interface as built-in modules (name, target_types, requires_keys, validate, async run)
Manifest	Declarative metadata (name, version, api_version, author, entry point) validated against a schema before load
Key gating	required_keys honoured by the planner: a key-gated plugin is skipped cleanly when its key is absent
State	Enable/disable persisted in the plugins table; disabled plugins are not imported
Isolation	Import failures are contained per plugin; version-checked imports guard API compatibility
Reference plugins	Wayback (key-free, CDX history) and VirusTotal (key-gated domain report), plus a documented starter template

Table 34. Plugin SDK mechanisms.

The module context given to every collector — built-in or plugin — provides the HTTP client, the TTL cache with single-flight, the offload pool, API-key resolution, and configuration lookup. Plugin authors therefore inherit the platform's resilience and politeness properties automatically.

13 Security and Privacy

13.1 Threat model

The threat model treats everything beyond the application boundary as untrusted, including responses from the targets the user chooses to scan. Eleven threats are tracked, each with a concrete mitigation.

#	Threat	Mitigation
T1	Hostile target payload (huge HTML, decompression bombs, malformed cert/EXIF/XML)	Size caps, timeouts, safe parsers, offloaded parsing; no eval/pickle
T2	XSS / script injection into reports	Jinja2 auto-escaping; escaped raw values; CSP meta tag
T3	Secret leakage	Logger redaction; vault as sole key source; reports exclude secrets
T4	Local theft of database / keys	0600/0700 permissions; Fernet encryption; OS keyring when available
T5	Supply-chain compromise	Minimal pinned dependencies; stdlib-first; version-checked plugin imports
T6	Runaway resource use	Pool limits, per-host semaphores, bounded retries, scheduler overlap prevention
T7	SSRF via redirects to internal hosts	Loopback/link-local/metadata ranges blocked by default; explicit opt-in
T8	Rate-limit / terms-of-service violations	Global and per-host limits; robots.txt respect; polite key-gating
T9	Plugin escape	Manifest gating; isolated import failure; explicit enablement
T10	AI prompt exfiltration	AI disabled by default; opt-in with explicit endpoint disclosure
T11	Audit gap	Local scan history; logged settings and key changes

Table 35. Threat summary (details in docs/threat_model.md).

13.2 Secrets management

API keys are stored Fernet-encrypted in the database (ciphertext plus a display hint), never in settings.json or the environment file after import. The Fernet master key is resolved in precedence order: an explicit environment variable, the OS keyring (when the optional keyring package is installed), then a 0600 key file. Runtime API-key resolution follows environment > vault > config, and the settings writer never serialises the api_keys section to disk.

13.3 Privacy posture

Principle	Implementation
Local-first storage	Targets, scans, findings, cases, keys, logs, and cache all reside in user-controlled local paths
No telemetry	No identifying telemetry exists in the codebase; the update check sends version metadata only
No account	The product has no user accounts or licensing server
Opt-in egress only	Scans send normal protocol traffic for the researched indicator; key-gated providers require an explicit key; the AI endpoint requires explicit opt-in; all egress can be routed through a configured proxy
Full removal	Deleting the data directory, config/, logs/, and cache/ removes all state

Table 36. Privacy principles and their implementation.

The ethical boundary is explicit: only publicly available information and authorised APIs; no exploitation, credential attacks, or authentication-bypass code; robots.txt is respected during profile checks; and scanned personal data (such as public WHOIS contacts) remains the operator's responsibility for lawful handling.

14 Quality Assurance and Test Results

14.1 Quality gate

Every change is gated on four checks passing together — formatting, linting, strict static typing, and tests. A phase or commit is accepted only when the entire gate is green. The same gate runs in CI on both supported Python versions before merge, and again in the release workflow before any artefact is published.

```
black . && ruff check . && mypy . && pytest -p no:cacheprovider -rN
```

Figure 8. The mandatory quality gate.

14.2 Test strategy

The strategy is layered: unit tests for pure logic; module tests with mocked HTTP for every collector; database tests covering migrations and repositories; CLI tests via Typer's runner; GUI tests using pytest-qt; and integration tests, including a fully offline end-to-end scan, marked so CI can include or exclude them deliberately. Coverage is enforced with fail_under=80 using branch coverage over core, modules, reports, and plugins.

Suite	Representative coverage	Focus
Core	engine, cache, config, correlation, risk, scheduler, vault, offload, worker pool	Engine pipeline, pure logic, concurrency primitives
Modules	tests/modules/** (per collector), http_client	Finding correctness, HTTP behaviours, caching, concurrency caps
Database	migrations, repositories	Forward-only migrations; typed repository contracts
CLI	full command surface	Command wiring, output, exit codes
GUI	qtbot, smoke, viz	Widget construction and interactions (offscreen Qt)
Integration	offline end-to-end scan	Full pipeline against loopback mocks — no internet
Assistant / Reports / Plugins	provider selection, golden report, loader edge cases	Deterministic narrative, output drift, fail-soft loading
Performance	performance budget, bench smoke	Regression budgets on core operations

Table 37. Test suite map.

14.3 Measured results

The following are the recorded release metrics for v1.0.0, taken from the project's final verification run.

81	756	89.54%	80%
Test files	Tests passing	Branch coverage	Coverage gate

Metric	Value	Notes
Test files	81	Across core, modules, database, CLI, GUI, integration, assistant, reports, plugins, performance
Tests passing	756	Full suite green under the quality gate
Branch coverage	89.54%	Over core, modules, reports, plugins — 9.5 points above the 80% floor
Coverage gate	fail_under = 80	Enforced in pytest configuration and CI
Integration tests	Offline E2E	End-to-end scan against loopback mocks; no internet dependency
Drift detection	Golden-file tests	Canonical report output pinned (tests/reports/golden/report.json)
Regression budgets	Performance tests	Core-operation budgets fail the build on regression
CI matrix	Python 3.13 / 3.14	Quality gate executed on both versions on every push and pull request

Table 38. Verification results for release v1.0.0.

Interpretation

Coverage is measured as branch coverage on the analytically critical packages (core engine, collectors, reporting, plugins) rather than inflating the figure with UI glue. The offline integration test proves the entire pipeline — classification, planning, concurrent execution, correlation, risk scoring, persistence, and export — works end-to-end without network access, which is precisely what makes the suite reliable in CI.

15 Performance Engineering

Performance work targeted four axes under a strict 'no unboundedness' policy: connection pooling, cache integration, retry logic, and offloading. A reproducible offline benchmark baseline was recorded on Linux with Python 3.14.

Section	Metric	Value
Cache	Operations per second	2,550,909
Cache	Hit rate	1.00
Engine	Scans per second (8 modules/scan)	7,414
Offload	Operations per second (4 workers)	15,344

Table 39. Benchmark baseline (same-host comparability indicators; absolute values vary by machine).

Key	Default	Meaning
network.pool_limit / per_host	100 / 8	Max open and per-host connections
network.max_concurrency / max_per_host	16 / 4	Global and per-host in-flight caps
network.rate_limit / per_host	0 (off)	Optional requests-per-second limits
network.backoff_base / backoff_max	0.5 s / 30 s	Retry delay growth and cap
network.http_cache	true	ETag / conditional response caching
performance.offload_workers	4	Bounded per-scan offload pool
cache.ttl / max_size	300 s / 1024	Cache expiry and size
scan.max_workers / default_timeout	8 / 30 s	Worker concurrency and per-task timeout

Table 40. Concurrency and performance configuration defaults.

Ordering within the HTTP client is deadlock-free: every request acquires the global slot before the per-host slot before the session call. Semaphores are per-HttpClient (per scan), so parallel scans never share or exhaust one another's budgets.

16 Packaging, Distribution, and CI/CD

16.1 Distribution channels

Artefact	Builder	Use case
Wheel + sdist	python -m build	pip install intelxtract
CLI binary	PyInstaller (intelxtract-cli.spec)	Standalone Linux x86_64; no Python required
GUI binary	PyInstaller (intelxtract-gui.spec)	Standalone desktop client
Docker image	Dockerfile (multi-stage) / compose	Reproducible headless runtime; GUI via display forwarding; app, GUI, and scheduler services
Checksums	sha256sum / sha512sum	Integrity verification on every release

Table 41. Distribution artefacts.

The multi-stage Dockerfile keeps the runtime image minimal and headless (around 150 MB); the compose stack adds a GUI service and a long-running scheduler service with named volumes for data, configuration, and plugins. Build scripts (build.sh / build.ps1) automate clean, cli-only, gui-only, wheel, and docker builds and emit checksums. System dependencies are explicit: PDF export requires WeasyPrint plus Pango/Cairo/GDK-Pixbuf/fontconfig, and the GUI requires Qt6 runtime libraries.

16.2 Continuous integration and release

Aspect	CI workflow	Release workflow
Trigger	Push / PR to main	Tag v*
Runner / matrix	ubuntu-latest; Python 3.13 and 3.14	ubuntu-latest
Steps	ruff, black --check, mypy, pytest (integration excluded) with coverage	Quality gate, wheel/sdist, PyInstaller CLI, GUI, binary smoke tests, checksums, draft GitHub Release
GUI testing	Qt system libraries installed for offscreen pytest-qt runs	—
PDF validation	—	WeasyPrint system libraries installed so PDF generation is exercised
Smoke tests	—	CLI binary verified: --version, doctor, and a real scan with JSON output
Permissions / timeout	—	contents:write, packages:write; 30-minute timeout

Table 42. CI and release pipeline configuration.

Releases are published as drafts with generated notes and checksums. Dependency versions are pinned with == throughout the runtime set, satisfying the supply-chain objective. The package exposes two console entry points — intelxtract (CLI) and intelxtract-gui (desktop) — and declares GPL-3.0-or-later licensing with requires-python >= 3.13.

```
flowchart  CI:  push/PR -> ruff -> black --check -> mypy -> pytest (coverage)
flowchart  REL: tag v* -> quality gate -> wheel/sdist
                -> PyInstaller CLI -> GUI -> binary smoke tests
                -> checksums -> draft GitHub Release
```

Figure 9. CI/CD flow summary.

17 Development Methodology

Development followed an iterative phase-gate process across 29 numbered phases (0–29). Each phase had a defined deliverable and an explicit acceptance gate, closed only when the full quality gate passed. This made progress measurable and kept regression risk bounded. The phases progressed from scaffolding and core foundations, through

the engine and persistence, to correlation, risk, reporting, CLI, GUI, cases, assistant, vault, performance, testing, documentation, packaging, and the v1.0.0 release.

Phase block	Theme	Representative deliverables
0–3	Scaffolding and core foundations	Package layout, config, logging, models, entities, HTTP client, cache
4–9	First collectors and the engine	Domain/IP collectors, registry, input engine, worker pool, scan engine
10–12	Persistence	SQLite schema, repositories, forward migrations
13–18	More modules and analysis	Email/username/website collectors; correlation; risk; reporting
19–21	Interfaces	Full CLI surface; PySide6 GUI
22–25	Operations and hardening	Cases; AI assistant; encrypted vault; performance
26–29	Assurance and release	Coverage gate and offline E2E; documentation; packaging; v1.0.0

Table 43. Development phase blocks (29 gated phases, all completed).

18 Known Limitations and Roadmap

The following limitations are documented transparently. Most are deliberate product or ethical boundaries rather than unfinished work.

Area	Current limitation	Planned enhancement
Hash targets	HASH is classified but has no collector	Key-gated hash-reputation module
IP reputation scope	DNSBL limited to Spamhaus ZEN; IPv6 out of DNSBL scope	Additional DNSBLs; IPv6 support
Port scanning	Active service/port identification intentionally omitted	Only if it can be kept passive and authorised
API surface	No REST API	Optional self-hosted REST API
Collaboration	Single-user, local database	Optional self-hosted shared cases
Concurrency store	Semaphores are per-process	Distributed rate limiting for multi-worker deployments
Assistant providers	Remote path is generic OpenAI-compatible	Provider-specific adapters and richer prompts
PDF rendering	Depends on WeasyPrint system libraries	Bundled renderer or alternate backend
Anonymity	No traffic anonymisation	Optional Tor/proxy management
Version metadata	Default app.version lags the canonical constant	Unify version sourcing across modules

Table 44. Known limitations and planned enhancements.

> **Design constraint, not a gap**
> The absence of any offensive capability — no exploitation, credential attacks, or authentication bypass — is an ethical design constraint. IntelXtract is intentionally limited to passive, authorised reconnaissance of publicly available information.

19 Conclusion

IntelXtract demonstrates that a local-first, account-free OSINT platform can deliver the breadth of commercial tooling without its privacy, cost, and cloud dependencies. The production release unifies 19 reconnaissance collectors, an asynchronous scan engine with bounded concurrency, an entity-centric correlation graph, a 22-rule deterministic risk model, case management, scheduling with change detection, and five-format reporting behind both a command-line and a desktop interface — all persisted in a user-controlled local database.

Quality was treated as a delivery requirement, not a phase-end activity. Every increment was accepted only under a uniform gate (formatting, linting, strict static typing, and tests) on Python 3.13 and 3.14. The released system passes 756 automated tests at 89.54% branch coverage, includes a fully offline end-to-end integration test and golden-file drift detection, and ships with a reproducible benchmark baseline and an automated release pipeline producing wheel and sdist, standalone binaries, a Docker image, and checksummed GitHub releases.

Correctness and safety are first-class properties: the risk engine is a pure function, the correlation graph is byte-stable, secrets are Fernet-encrypted and redacted from logs, report rendering is auto-escaped, hostile-response defences are documented in a formal threat model, and scanning is constrained to passive, authorised sources. The product meets its objectives of breadth, determinism, extensibility, and privacy while documenting its limits honestly, and its modular architecture and plugin SDK leave clear room for the planned enhancements without compromising the local-first guarantees that define it.

A Appendix — Reference Data

A.1 Runtime dependency versions

Package	Version	Purpose
aiohttp	3.14.4	Async HTTP client
aiosqlite	0.22.1	Async SQLite driver
typer	0.27.2	CLI framework
rich	15.0.0	Terminal formatting
jinja2	3.1.6	HTML report templating
PySide6	6.10.3	Qt 6 GUI framework
cryptography	50.0.2	Fernet encryption (key vault)
networkx	3.7	Correlation graph analysis
Pillow	12.3.0	Image EXIF metadata
pypdf	6.20.0	PDF metadata

Table 45. Pinned runtime dependency versions.

Optional extras: weasyprint (PDF rendering; graceful HTML fallback when absent), keyring (OS keyring vault backend). Development and build: ruff 0.16.10, black 26.10.0, mypy 2.4.0, pytest 9.1.1, pytest-asyncio 1.4.0, pytest-cov 7.1.0, pytest-qt 4.4.0, pyinstaller 6.22.3, build 1.2.2.

A.2 Default configuration

Key	Default
app.name / app.version	IntelXtract / 1.0.0
scan.default_mode / default_timeout	quick / 30.0 s
scan.max_workers	8
cache.enabled / ttl / max_size	true / 300.0 s / 1024
network.pool_limit / per_host	100 / 8
network.max_concurrency / max_per_host	16 / 4
network.backoff_base / backoff_max	0.5 s / 30 s
performance.offload_workers	4
logging.level / file	INFO / logs/intelxtract.log
Environment prefix / config path	INTELXTRACT_ / .env, config/settings.json

Table 46. Default configuration values.

A.3 Repository layout

```
core/        engine, input engine, scheduler, worker pool, cache,
             http client, correlation, risk, vault, plugin loader
modules/     domain/ ip/ website/ email/ username/ certificate/
             metadata/ news/  (19 collectors) + base classes
database/    models, schema.sql, migrations, repositories, persistence
reports/     builder, exporters (JSON/CSV/MD), brand tokens
templates/   report.html.j2 (Jinja2, auto-escaped)
cli/         Typer command surface (scan, report, history, ...)
gui/         PySide6 app: pages/, viz/, bridge, context, theme
assistant/   providers (disabled / deterministic / remote)
plugins/     SDK contract + Wayback + VirusTotal + starter template
tests/       core modules database cli gui integration assistant
             reports plugins  (81 files, 756 tests)
scripts/     bench.py, build.sh, build.ps1, pyinstaller specs
docs/        architecture, guides, plugin SDK, threat model, privacy
```

A.4 Verification and reproduction commands

```
# Quality gate (lint, format, type-check, test)
black . && ruff check . && mypy . && pytest -p no:cacheprovider -rN

# Benchmark baseline
python scripts/bench.py --json

# Run a scan and export every format
intelxtract scan example.com --mode deep
intelxtract report 1 --format html --out exports/report.html
intelxtract report 1 --format pdf  --out exports/report.pdf
intelxtract report 1 --format json --out exports/report.json

# Environment health
intelxtract doctor
```

End of report — IntelXtract Engineering Project Report v1.0 (IX-ENG-REP-1.0). All quantitative claims are traceable to the repository source tree, the automated test suite, and the project documentation.